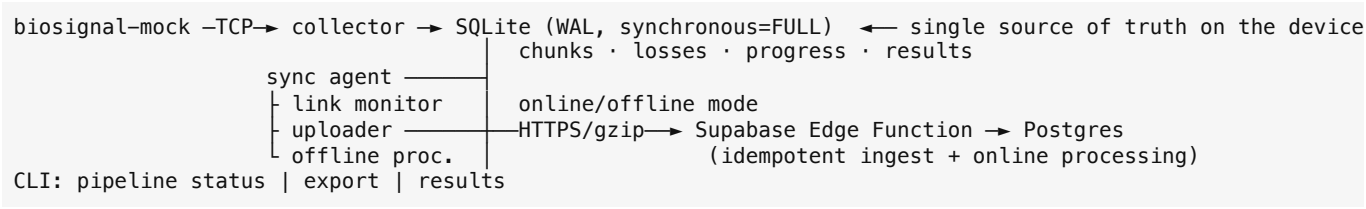
Design note: Never Lose a Heartbeat

Architecture



There are two processes, each a systemd service with `Restart=always`:

- **Collector.** Its only job is to get every sample from the mock onto disk. It never touches the network. Every request carries `from_index` set to the next index it still needs. Retrying after a crash, a timeout, a corrupt reply or a duplicate is therefore always "ask again for exactly what is missing".
- **Sync agent.** It runs the link monitor, uploads data (live and catch-up), and processes windows on the device while offline. If it is slow, crashes or loses the network, the collector is not affected.

Device storage invariant. For each (session, sensor), the saved chunks and the recorded losses tile `[0, next_index)` exactly once. Each mock reply is saved in one transaction, and the collector asks for the next data only after that commit has been fsynced.

Cloud. The cloud is a Supabase Edge Function (Deno/TypeScript) backed by Postgres, all on the free tier.

- `/ingest` inserts with `ON CONFLICT DO NOTHING` on `(session, sensor, first_index)`.
- An advisory lock on the session serialises processing.
- The ingest request that completes a window also computes its result, so cloud results need no separate polling worker.

Processing. The logic is implemented once in Python (on the device) and once in TypeScript (in the cloud). A golden test (`tests/golden_cases.json`, 80 windows with gaps) requires the two to give identical output.

- The peak detector is streaming. It uses a sliding-window maximum and keeps state (`scan_next`, `last_peak`, `peaks`) across windows. This makes the "previous peak" rule exact over the whole session.
- A window is processed only once every sample it depends on is *decided* (saved or recorded as lost). This includes the sample after the window, which the peak rule needs.

How faults are handled

Fault	Handling
Corrupt reply	The CRC32 is checked over the raw `data` text exactly as sent. A reply that fails is discarded and the same `from_index` is requested again.
Duplicate reply or overlap	Anything below `next_index` is dropped.
Mock silent	After 3 s the collector reconnects and resumes from `next_index`.
Sensor dropout / buffer overflow	Ranges in the mock's `lost` are stored with the mock's reason. An unexplained gap becomes a `not_delivered_by_mock` loss only after **two separate explicit requests** for that index were answered past it (a duplicated reply counts once). After 3 unusable replies in a row the collector sends a plain read to resync with whatever session the mock is in.
Mock restart	A new `session_id` closes the old session. Its unread tail is recorded as lost with `to_index: null`.
Process kill	systemd restarts the process within 1 s, and it resumes from `next_index`. The mock still holds the last 10 s, so nothing is lost.
Network cut	A failed upload (hard 4 s deadline) confirmed by a failed 2.5 s health check switches the device to offline mode (measured **6.9 s**). Offline windows are processed on the device. 2 successful health checks switch back to online mode (measured **3.0 s**).
Catch-up	The upload backlog is sent oldest first, in batches of up to 30 s of data per sensor, gzip-compressed. A per-sensor watermark (`cloud_next`) advances only after the cloud acknowledges the batch (one small row update, not a flag per chunk, to keep disk writes low), and a retry is harmless.
Disk full	A 32 MB reserve file is released first. If the disk is still full, data is not acknowledged. The mock then reports a buffer overflow, and that loss is recorded with the reason "(device disk was full)".
Clock change	No logic uses the wall clock. All timers use monotonic time, and data is ordered by sample index. Wall time is stored only to report latency.

What is guaranteed

- Every sample the collector accepted (with a valid CRC) is on disk exactly once and unchanged, and survives `kill -9`, reboot and power loss once committed.
- Every index of a session is either saved or listed as lost with a reason. Gaps are never filled with values.
- The cloud ends up with exactly the device's copy, losses included. This is idempotent upload plus at-least-once retry, which gives exactly-once storage.
- Results are computed with identical logic online and offline.

What is not guaranteed

- **Data the mock never delivered.** Examples: a mock restart, the mock's 10 s buffer overflowing while the device was down, a full disk lasting longer than 10 s. This data is recorded as lost, not recovered.
- **The 5 s result latency when the cloud itself is slow** (for example an Edge Function cold start). If a window has not been acknowledged by the cloud 4 s after it is ready, the device also produces an offline result for it.
- **During catch-up after an outage**, live windows can get an offline (device) result first. The cloud fills in the online results as the backlog drains.

Maximum data loss on a power cut

≤ 1 second per sensor. Each reply holds at most 1 s of data and is committed with `synchronous=FULL` (fsync of the WAL) before the next request.

What can be lost is limited to:

- the reply in flight;
- data the mock had buffered but not yet sent. The collector polls every 0.5 s, so this is about 0.5–1 s in practice.

Because the mock runs on the same board, it also loses power and restarts with a new session. The collector records the old session's unread tail as `to_index: null`, so the loss is listed, never silent.